

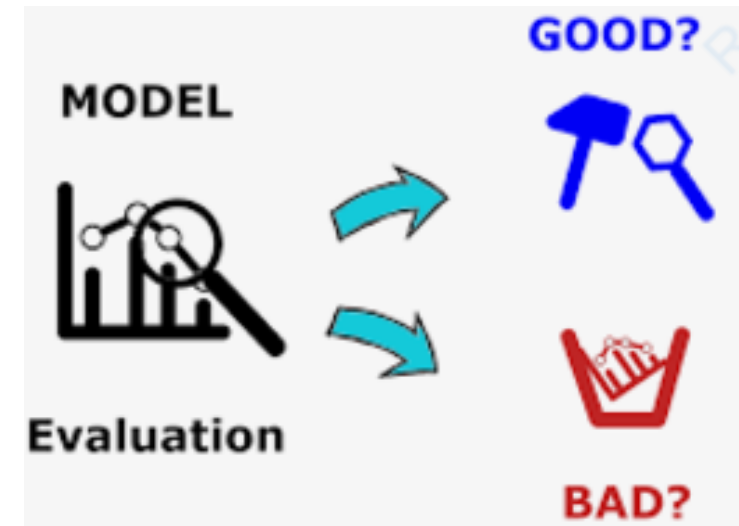


Evaluation Metrics for Classification

Rasha Obeidat, PhD
Deep Learning

Model Evaluation

- Evaluation in classification refers to the process of **quantitatively assessing the performance of a classifier** to determine how well it predicts the target outcomes.
- It involves measuring the classifier's ability to correctly distinguish between different classes using predefined metrics.
- Examples:
 - Accuracy
 - Recall
 - Precision
 - F1-score



Why is Evaluation Important?

Measure Effectiveness: Evaluation helps us determine whether a classifier is effective in solving the intended task.

Compare Models: It enables comparison between different classifiers or configurations to select the best-performing model.

Understand Limitations: Evaluation reveals the weaknesses of a model, such as biases, errors, or poor performance on specific classes.

Guide Improvements: By identifying areas where the model struggles, evaluation helps in refining and improving the model.

Ensure Practical Utility: Evaluation ensures the model's predictions meet the required performance thresholds for real-world deployment.

Validation vs Testing in Model Evaluation

- **Validation**

- **Purpose:** Used during training to tune hyper-parameters and monitor progress.
- **Role:** Guides model adjustments and helps select the best model configuration.
- **Usage:**
 - Hyper-parameter tuning.
 - Early stopping to avoid overfitting.
- **Timing:** During training.

- **Testing**

- **Purpose:** Used after training to evaluate the final model's performance.
- **Role:** Assesses the model's generalization ability on unseen data.
- **Usage:** Reporting final performance metrics (e.g., accuracy, F1-score).
- Comparing models for deployment or publication.
- **Timing:** After training is complete

Accuracy

- **Accuracy** is the percentage of the instances that have been classified correctly.

- Accuracy =
$$\frac{\# \text{ correct classified examples}}{\text{Total \# of Examples}}$$

- Example:

Correct class	C1	C1	C1	C2	C2	C3	C3	C3	C3	C3	C3	C3
Predict class	C1	C2	C3	C2	C3	C3	C2	C1	C3	C3	C3	C1

Accuracy

- **Accuracy** is the percentage of the instances that have been classified correctly.

- Accuracy =
$$\frac{\# \text{ correct classified examples}}{\text{Total \# of Examples}}$$

- Example:

Correct class	C1	C1	C1	C2	C2	C3	C3	C3	C3	C3	C3	C3
Predict class	C1	C2	C3	C2	C3	C3	C2	C1	C3	C3	C3	C1

$$\text{Accuracy} = \frac{6}{12} = 0.5$$

Accuracy

- What is the problem with accuracy?
 - Accuracy = $\frac{\# \text{ correct classified examples}}{\text{Total \# of Examples}}$
- Accuracy *can* be a useful measure if we have the dataset have balanced classes. I.e. classes have similar amounts of examples.
- If dataset has a skewed class distribution, accuracy *can* be not useful at all. High accuracy may not indicate a good performance, e.g., the model that always predict the highly populated class.

Correct class	C1	C2	C3	C3	C3	C3	C3	C3	C3	C3	C3	C3
Predict class	C3	C3	C3	C3	C3	C3	C3	C3	C3	C3	C3	C3

- Accuracy = $\frac{10}{12} = 0.8333$

Accuracy

- A test can have a high accuracy but actually perform worse than a test with a lower accuracy.
- Model 1: the predictions include all classes:

Correct class	C1	C2	C3	C3	C3	C3	C3	C3	C3	C3	C3	C3
Predict class	C1	C2	C1	C1	C2	C2	C3	C3	C3	C3	C3	C3

- $\text{Accuracy} = \frac{8}{12} = 0.666$

- Model2: all of the predictions belong to one class:

Correct class	C1	C2	C3	C3	C3	C3	C3	C3	C3	C3	C3	C3
Predict class	C3	C3	C3	C3	C3	C3	C3	C3	C3	C3	C3	C3

- $\text{Accuracy} = \frac{10}{12} = 0.8333$

- Model 1 is better than model 2 even it has less accuracy value.

Confusion Matrix for Binary Classification

- A **confusion matrix** is a table that is used for describing and summarizing the performance of a classification algorithm on a set of test data for which the true values are known.

Actual class \ Predicted class	Class 1	Class 0	Total
	Class 1 (positive)	Class 0 (negative)	Total
Class 1 (positive)	TP	FN	TP+FN
Class 0 (negative)	FP	TN	FP+TN
Total	TP+FP	FN+TN	

TP: number of samples belonging to the positive class and correctly assigned to it.

FP: number of samples that doesn't belong to the positive class and incorrectly assigned to it.

FN: number of samples belong to the positive class and incorrectly assigned to the negative class.

TN: number of samples that doesn't belong to the positive class and correctly assigned to it.

Precision/Recall/F-score

- **Precision** is the ratio of positive prediction that are correct.

$$P = \frac{TP}{TP + FP}$$

- **Recall** is the ratio of the positive examples that have been correctly predicted.

$$R = \frac{TP}{TP + FN}$$

- **F-score** is the harmonic mean of precision and recall.

$$F = \frac{2 \times P \times R}{P + R}$$

- Using confusion matrix, **accuracy** can be computed as:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Why F-score?

- To summarize precision and recall into a single value, the **F-score** is used instead of simple mean.
- **F-score** is the harmonic mean of precision and recall..

$$F = \frac{2 \times P \times R}{P + R}$$

- The harmonic mean is preferred over a simple average because it *penalizes extreme values of precision or recall*. This ensures a balance between the two metrics
- For example, a classifier with a precision of 1.0 and a recall of 0 has a simple average of 0.5, but its **F1-score** is 0, highlighting the failure of the classifier in terms of recall.

Confusion Matrix for Binary Classification

Correct class	c_1	c_1	c_1	c_1	c_0	c_0	c_0	c_0	c_0	c_0	c_0	c_0
Predict class	c_1	c_0	c_0	c_1	c_1	c_0	c_0	c_0	c_1	c_0	c_1	c_0

Actual class \ Predicted class	Predicted class		Total
	c_1	c_0	
c_1	2	2	4
c_0	3	5	8
Total	5	7	12

Typically, the **minority** class is considered as the positive class. E.g., to evaluate a spam detection model, spam email are considered as positive.

Confusion Matrix for Binary Classification

Correct class	c_1	c_1	c_1	c_1	c_0	c_0	c_0	c_0	c_0	c_0	c_0	c_0
Predict class	c_1	c_0	c_0	c_1	c_1	c_0	c_0	c_0	c_1	c_0	c_1	c_0

Predicted class \ Actual class	c_1	c_0	Total
c_1	2	2	4
c_0	3	5	8
Total	5	7	12

$$P = \frac{TP}{TP+FP} = \frac{2}{5}$$

$$R = \frac{TP}{TP+FN} = \frac{2}{4}$$

$$F = \frac{2 \times \frac{2}{5} \times \frac{2}{4}}{\frac{2}{5} + \frac{2}{4}} = \frac{4}{9}$$

Confusion Matrix for Multi-class Classification

TP: number of samples belonging to a particular class and correctly assigned to that class.

FP: number of samples that doesn't belong to a particular class and incorrectly assigned to that class.

FN: number of samples belong to a particular class and incorrectly assigned to another class.

TN: number of samples that doesn't belong to a particular class and correctly assigned to that class.

Confusion Matrix for Multi-class Classification

Correct class	c_1	c_1	c_1	c_1	c_2	c_2	c_3	c_3	c_3	c_3	c_3	c_3
Predict class	c_1	c_1	c_1	c_3	c_2	c_3	c_3	c_1	c_1	c_3	c_3	c_3

Predicted class \ Actual class	c_1	c_2	c_3	Total
c_1	3	0	1	4
c_2	0	1	1	2
c_3	2	0	4	6
Total	5	1	6	12

Confusion Matrix for Multi-class Classification

Predicted class \ Actual class	c_1	c_2	c_3	Total
c_1	3	0	1	4
c_2	0	1	1	2
c_3	2	0	4	6
Total	5	1	6	12

Class	TP	FN	FP
c_1	3		
c_2			
c_3			

Confusion Matrix for Multi-class Classification

Predicted class \ Actual class	c_1	c_2	c_3	Total
c_1	3	0	1	4
c_2	0	1	1	2
c_3	2	0	4	6
Total	5	1	6	12

Class	TP	FN	FP
c_1	3	1	
c_2			
c_3			

Confusion Matrix for Multi-class Classification

Predicted class \ Actual class	c_1	c_2	c_3	Total
c_1	3	0	1	4
c_2	0	1	1	2
c_3	2	0	4	6
Total	5	1	6	12

Class	TP	FN	FP
c_1	3	1	2
c_2			
c_3			

Confusion Matrix for Multi-class Classification

Predicted class \ Actual class	c_1	c_2	c_3	Total
c_1	3	0	1	4
c_2	0	1	1	2
c_3	2	0	4	6
Total	5	1	6	12

Class	TP	FN	FP
c_1	3	1	2
c_2	1		
c_3			

Confusion Matrix for Multi-class Classification

Predicted class \ Actual class	c_1	c_2	c_3	Total
c_1	3	0	1	4
c_2	0	1	1	2
c_3	2	0	4	6
Total	5	1	6	12

Class	TP	FN	FP
c_1	3	1	2
c_2	1	1	
c_3			

Confusion Matrix for Multi-class Classification

Predicted class \ Actual class	c_1	c_2	c_3	Total
c_1	3	0	1	4
c_2	0	1	1	2
c_3	2	0	4	6
Total	5	1	6	12

Class	TP	FN	FP
c_1	3	1	2
c_2	1	1	0
c_3			

Confusion Matrix for Multi-class Classification

Predicted class \ Actual class	c_1	c_2	c_3	Total
c_1	3	0	1	4
c_2	0	1	1	2
c_3	2	0	4	6
Total	5	1	6	12

Class	TP	FN	FP
c_1	3	1	2
c_2	1	1	0
c_3	4		

Confusion Matrix for Multi-class Classification

Predicted class \ Actual class	c_1	c_2	c_3	Total
c_1	3	0	1	4
c_2	0	1	1	2
c_3	2	0	4	6
Total	5	1	6	12

Class	TP	FN	FP
c_1	3	1	2
c_2	1	1	0
c_3	4	2	

Confusion Matrix for Multi-class Classification

Predicted class \ Actual class	c_1	c_2	c_3	Total
c_1	3	0	1	4
c_2	0	1	1	2
c_3	2	0	4	6
Total	5	1	6	12

Class	TP	FN	FP
c_1	3	1	2
c_2	1	1	0
c_3	4	2	2

Macro-Averaging vs Micro-Averaging

- Once we computed the confusion matrix and the TP,FP,FN for every class, the individual results can be combine using two ways:
 1. **Macro-averaging:** the values for precision, recall and F1 score are calculated separately per class, and then averaged to get the *macro-averaged precision, recall and F-scores*.
 2. **Micro-averaging:** we compute the overall TP as the sum of the individual classes' TP values, same thing for FN and FP. Then the *micro-averaged recall, precision* are computed from the overall TP,FP and FN, and the *micro-averaged F-score* is simply the harmonic mean of the micro-averaged precision and recall.

Computing Micro-averaged and Macro-averaged metrics

Class	TP	FN	FP
c_1	3	1	2
c_2	1	1	0
c_3	4	2	2
Total	8	4	4

Class	Precision	Recall	F-score
c_1	0.6	0.75	0.666
c_2	1.0	0.5	0.666
c_3	0.666	0.666	0.666
Macro -averaged	0.755	0.638	0.666

- Micro-averaged P = $\frac{TP_{total}}{TP_{total} + FP_{total}} = \frac{8}{12} = 0.666$

- Micro-averaged R = $\frac{TP_{total}}{TP_{total} + FN_{total}} = \frac{8}{12} = 0.666$

- Micro-averaged F = $\frac{2 \times \frac{8}{12} \times \frac{8}{12}}{\frac{8}{12} + \frac{8}{12}} = \frac{8}{12} = 0.666$

Micro-averaged vs Macro-averaged Metrics

- **Macro-averaged** metrics gives equal weights(importance) to all classes, so it gives a clear picture of the classifier ability to recognize the rare classes (classes with few examples) when the distribution of the dataset is skewed.
- **Micro-averaged** metrics gives equal weights(importance) to all examples regardless the classes they belong to, so it can be overwhelmed by the classifier performance on small classes (highly populated classes)
- It is highly likely to have a clear difference between macro-averaged and micro-averaged scores when the dataset is imbalanced and the classifier can't recognize most of the rare classes.
- In multiclass classification, if the test data has examples from all classes, then the *accuracy*, the *Micro-averaged precision*, *recall* and *f-scores* are the same.

Precision, recall, and F-score in sklearn

- In Python's **sklearn** library, we can easily calculate the precision and recall for each class in a multi-class classifier using the function **sklearn.metrics.classification_report**.
- See this link for an example: <https://towardsdatascience.com/multi-class-metrics-made-simple-part-i-precision-and-recall-9250280bddc2>

Other Evolution Metric

- Accuracy, Recall, Precision are among the mostly common evaluation matrices used in machine learning, however , different task have different metrics based on the nature of the task.
- Other metrics that can be used in Classification problems like the Log loss, AUC-ROC Curve, F-Beta , BLUE score etc.

- These slides are the intellectual property of **Dr. Rasha Obeidat®**.
- The slides may **not be modified or redistributed** without explicit permission.
- **Contact:**
 - Obeidat.rm@gmail.com
 - Rmobeidat@just.edu.jo

